CSC 312: Software Design
Final Report

Group Information

Group Number: 1

Member List: Nikita Aleksii, Robby Votta, Yurda Yolcu, and Maria Fajardo.

Introduction

AssemblyViz is a web-based simulator and code editor for the HYMN and RISC-V instruction set architectures (ISA), designed to replace and extend simHYMN (https://cburch.com/socs/hymn/index.html), the tool currently used by Davidson College's Mathematics and Computer Science department in its Computer Organization course (CSC 250). The project's primary goal is to provide a more intuitive interface for writing and stepping through assembly code, while extending simHYMN's functionality with line-specific debugging, memory cell history, and support for a real-world ISA.

The motivation for AssemblyViz came directly from the observed need for the existing tool. Students in CSC 250 struggled with simHYMN's interface, and our primary customer, Dr. Hammurabi Mendes, identified this as a source of mental bias against assembly programming itself. simHYMN also offers no debugging support and no way to observe how memory cells evolve during execution, which made it difficult for students to trace data flow and for instructors to diagnose errors. Beyond the interface concerns, HYMN is a pedagogical ISA and does not expose students to architectures they will encounter in advanced coursework or industry. AssemblyViz addresses this by integrating RISC-V alongside HYMN, giving students continuity from classroom fundamentals to real-world assembly.

We followed a hybrid Scrum and Extreme Programming (XP) development process. This combination supported iterative delivery through sprint-based planning while preserving code quality through XP practices such as continuous testing.

Requirements

Requirement 1: Intuitive, user-friendly interface with clearly separated panels. Design a clean, logically partitioned interface that distinctly separates the code editor, memory cells, registers, input field, and output field so that users can locate and interact with each component without confusion.
Elicitation Method: Observation. In the Computer Organization class, we noticed that when students interacted with simHYMN, they struggled to distinguish between panels, lost track of register values while editing code, or confused the input and output fields.
Classification: Observable. Students did not explicitly request "separate the panels"; instead, the need was observed through their visible struggles and errors while navigating a cluttered or poorly organized interface.

Requirement 2: Web-based simulator accessible across operating systems. Move to a browser-based web application, eliminating the need for installations, so that any user with a web browser can access the simulator regardless of their operating system.
Elicitation Method: Interview with Dr. Hammurabi Mendes and students. When we asked about barriers to using the current tool, users reported issues with installing the application on MacOS. Since most students use MacOS, there is a natural need to switch to a web-based application.
Classification: Articulate. Users were able to directly and explicitly state this need – "I can't open it on my Mac," or "It should just work in a browser."

Requirement 3: Line-specific debugging with meaningful error messages. Provide a debugging feature that identifies the exact line number where an error occurs and displays a descriptive message indicating the type of error, enabling students to diagnose and correct their code efficiently.
Elicitation Method: Interviews and observation. During interviews, students expressed frustration with the lack of error reporting in the existing tool. Also, instructors repeatedly helped students manually trace through code to find bugs that a proper debugger would have flagged instantly.
Classification: Articulate. Students clearly communicated what was missing. They knew they wanted error messages tied to specific lines because they had experience with such features in higher-level languages.

Requirement 4: Memory history with change highlighting. Maintain and display a history of memory cell states across execution steps, visually highlighting which cells changed at each step so that students can trace data flow and understand how instructions modify memory.
Elicitation Method: Interview. Dr. Hammurabi Mendes wanted to incorporate this feature because students repeatedly rerun their programs to check whether a particular memory cell had changed. Because of that, students lost track of what changed and when.

Classification: Tacit. Students understood instinctively that tracking memory changes was important to their learning process, but they could not easily articulate it as a request.

Requirement 5: Execution of complex programs (50+ Lines) within 5 seconds. Execute HYMN and RISC-V programs containing 50 or more lines of code at the maximum allowable simulation speed, completing full execution within 5 seconds to maintain a responsive and productive user experience.
Elicitation Method: Prototype feedback. Dr. Terrence Lim, a professor at Davidson College who may teach the Computer Organization class in the future, explicitly emphasized the importance of running complex programs quickly. To validate and quantify this concern, we fed algorithms with varying code sizes into the prototype and measured execution times, establishing the threshold of 4.5 seconds.
Classification: Articulate. Dr. Terrence Lim directly and clearly communicated the need for the fast execution of complex programs.

Requirement 6: Incorporation of the RISC-V instruction set architecture. Support the RISC-V instruction set architecture alongside HYMN, allowing students to write, assemble, and execute RISC-V programs and thereby gain exposure to a real-world ISA used in industry and academics.
Elicitation Method: Interview with Dr. Hammurabi Mendes. He noted that the pedagogical HYMN language, while useful for fundamentals, does not prepare students for the ISA they will encounter in advanced coursework or industry.
Classification: Latent. Most students did not request RISC-V support because they might not yet know what RISC-V is or why it matters. The requirement stems from a need to bridge the gap between the classroom and the real world.

SMART User Story

User Story 1. As a student, I want a clearly partitioned interface separating the code editor, memory cells, registers, input field, and output field, so that I can locate any component within 5 seconds of opening the application.

- Acceptance Test: The interface displays all five components, such as the code editor, memory cells, registers, input field, and output field, with clearly separable boundaries. We tested the app with 10 users, all of whom were able to navigate the app easily within 5 seconds.

User Story 2. As a student, I want to execute assembly code in a browser-based application so that I don't have to deal with the issues related to my operating system.

- Acceptance Test: The application successfully runs on Google Chrome and Safari on a local machine.

User Story 3. As a professor, I want to debug students' erroneous code so that I can pinpoint the lines where an error occurred within one second and explain what the error was.

- Acceptance Test: The debugger successfully outputs the location of the code error and indicates whether an instruction is not defined or a label is misspelled.

User Story 4. As a student, I want to step through my program one instruction at a time, so that I can see which memory cells changed, highlighted after each step of execution.

- Acceptance Test: Changed memory cells are visually highlighted after each step, and previous values remain visible in a history panel.

User Story 5. As a professor, I want to run complex assembly algorithms with 50 or more lines in class so that I can show the code output to my students within 5 seconds.

- Acceptance Test: To evaluate software performance, we measured the time it takes to execute RISC-V code of different sizes: 5, 10, 20, 50, and 100 lines of code. The maximum running time is attributed to the 100-line code, taking 4.5 seconds to execute.

User Story 6. As a student, I want to write and execute RISC-V assembly programs so that I can gain familiarity with an industry-standard ISA.

- Acceptance Test: The simulator supports writing, assembling, and executing RISC-V programs, and students can run at least the core RV32I instructions.

Backlogs

Product Backlog (What does Dr. Mendes want?)

Completed

Backlog ID	Type	Description
0	feature	frontend framework and UI shell
1	feature	HYMN ISA definitions. 8 opcodes, 3 registers, 32 bytes of memory
2	feature	HYMN ISA definitions. 8 opcodes, 3 registers, 32 bytes of memory
3	feature	HYMN parser/assembler. tokenizes text, encodes symbol table
4	feature	RISC-V assembler. encodes all RV32I formats and pseudo-instructions
5	feature	HYMN machine simulator. fetch, decode, execute cycle
6	feature	RISC-V decoder. splits 32-bit words into opcode fields
7	feature	High-level HYMN executor wrapping machine.py
8	feature	Byte-addressable RISC-V memory model
9	feature	Stateless RISC-V simulation with state snapshots
10	testing	HYMN unit tests. N test files for K test functions
11	testing	RISC-V unit tests. N test files for K test functions
12	feature	Manual validation against SimHYMN reference programs (sumn, mult, primes, virus)
13	feature	RISC-V integration test using merge_sort.S
14	Bug fix	HYMN parser.py converted to a two-pass regular expression system
15	Bug fix	RISC-V assembler.py converted to a two-pass regex system
16	feature	React/TypeScript project scaffold connected to the backend
17	feature	3-panel layout: Code Editor, Results, Register Panel
18	feature	CodeEditor.tsx. editor with line numbers and Assemble button

19	feature	Navbar.tsx. play/pause, step forward/backward, reset, ISA toggle
20	feature	RegisterPanel.tsx. 3 HYMN registers and 32 RISC-V registers with ABI names
21	feature	ResultsPanel.tsx. assembled instruction listing with active row highlighting
22	feature	ISA toggle with instant full state reset
23	feature	Console output display for HYMN WRITE and RISC-V recall output
24	feature	Import .txt and export source/results buttons
26	feature	Toggle playback speed slider
27	test	Final integration testing. frontend to backend round-trip for both ISAs
28	feature	Console input display for HYMN READ. integer input queue
29	UI	Figma-sourced icons integrated (logo, play, forward, backward, reset)
30	UI	Cell change flash animation when memory or register value updates
31	UI	Active row highlighting in the Results panel tracking the current PC
32	UI	Pseudo-instruction expansion markers in Results listing
35	setup	Project repository, branching strategy, and development workflow established
36	setup	README files written for main repo, backend, frontend, HYMN, and RISC-V

Removed

Backlog ID	Type	Description
37	feature	Plain HTML/JS prototype replaced by React/TypeScript for component (script.js, styles.css) architecture and type safety

Not completed yet

Backlog ID	Type	Description	status	Acceptance Criteria
38	infra	CI/CD pipeline setup	incomplete	Tests run automatically on GitHub for automated test runs) after every push
39	infra	Containerization for deployment	incomplete	App can be launched in a container and served to end users
40	feature	Debugger with breakpoint support	Pushed back	Program pauses automatically at a specified memory address
41	feature	Inline error indicators in the code editor	Suggestion through feedback	Parse errors are highlighted on the specific line in the editor, not only in the status bar
42	feature	Onboarding walkthrough tutorial	Suggestion through feedback	A first-time user can complete a guided walkthrough of the interface without external documentation
43	feature	Example program library	Suggestion through feedback	Users can load a curated set of sample programs started (e.g., primes, sum, sort) from a menu

44	feature	Keyboard shortcuts for simulation controls	Suggestion through feedback	Play, step forward, step back, and reset are accessible via keyboard

The original plan was to use an HTML/JS prototype (frontend/script.js, frontend/styles.css), but this was dropped. Once the React/TypeScript implementation was agreed upon. This decision was made to support component-based architecture with TypeScript type safety. We have also not completed automatic testing, containerization, or deployment. This will be a later task for when this is actually implemented for CSC 250 classes. The main point of the project at this stage was to create a concept that works and later down the line can be better tested, approved by the department, and then deployed. We also had many great suggestions from presentations that we must implement before deployment.

Sprint Backlog

Sprint 1: Foundation & Architecture February 3 – February 18 (Proposal – Report 1)

Backlog Item ID	Backlog item type	Description	Status	Acceptance Criteria
0	setup	Set up project repository, branching strategy, and development workflow	completed	Developers can branch from the main code base to push changes as needed
1	feature	Draft frontend framework with basic UI shell and navigation structure via Figma	completed	Visual version of the product created
2	feature	Implement HYMN ISA definitions (instructions.py) 8 opcodes, 3 registers, 32 bytes of memory	completed	Opcodes, registers, and memory are clearly defined for other files to use
3	feature	Implement RISC-V ISA constants and opcode table (isa.py)	completed	Opcodes, registers, and memory are clearly defined for other files to use

4	feature	Implement HYMN parser/assembler (parser.py), which tokenizes text to encode the symbol table	completed	Successfully map every input line to an 8-bit opcode instruction
5	feature	Implement RISC-V assembler (assembler.py)	completed	Successfully encode all RV32I instruction formats and pseudo-instructions
6	feature	Implement HYMN machine simulator (machine.py)	completed	Successfully run fetch, decode, then execute cycle for each line of text
7	feature	Implement RISC-V decoder (decoder.py) to split 32-bit words into opcode fields	completed	Define what instructions go to which opcodes in a 32-bit line
8	feature	Implement high-level HYMN executor from simulator (executor.py) Step to the next line, load instruction, run it, and reset	completed	Successfully wraps machine.py to execute code in the form of a dictionary
9	feature	Implement byte-addressable RISC-V memory model (memory.py)	completed	Successfully determine whether an instruction needs to store the full 32-byte word, half of the word, or just a single byte
10	feature	Implement stateless RISC-V simulation with state snapshots (simulation.py)	completed	Return the correct and complete machine state after every step
11	testing	Write HYMN unit tests (5 test files, 160 test functions)	completed	Return all 160 correct test functions
12	testing	Write RISC-V unit tests (5 test files, 158 test functions)	completed	Return all 158 correct test functions
13	feature	Implement debugger with breakpoint support (debugger.py)	Incompete	Run a program and have it pause automatically at a specific memory

				address

Item 13 was removed from the backlog. Hymn has breakpoint debugging capabilities, but we decided to focus on core frontend functionality. This can be a future feature added before or after deployment.

Sprint 2: Assembler Implementation February 18 – March 20 (Report 1 – Report 2)

Backlog Item ID	Backlog item type	Description	Status	Acceptance Criteria
14	testing	Manual validation against SimHYMN reference sumn, mult, primes, virus programs	completed	Successfully runs every single step of the programs to the end, the exact same way SimHYMN would
15	testing	Write an integration test using merge_sort.S RISC-V program	completed	Successfully runs every single step of the programs to the end, the exact same way RISC-V logic would
16	Bug fix	Changed hymn parser.py to a regular expression two-pass system	completed	The pipeline now successfully identifies flags in first flags to be able to run test programs
17	Bug fix	Changed RISC-V assembler.py to a regular expression two-pass system	completed	The pipeline now successfully identifies flags in first flags to be able to run test programs
18	feature	Set up React/TypeScript project scaffold to backend	completed	…
19	result	Implement 3-panel layout: Code Editor (left)	completed	…
20	feature	Implement CodeEditor.tsx	completed	Fully functional assembly editor with line numbers and Assemble button

21	feature	Implement Navbar.tsx	completed	play/pause, step forward/backward, reset, ISA toggle
22	feature	Implement RegisterPanel.tsx	completed	HYMN (3 registers) and RISC-V (32 registers with names)
23	feature	Implement MemoryPanel.tsx	completed	memory viewer with hex, decimal, or instruction toggle display
24	feature	Implement ResultsPanel.tsx	completed	Assembled instruction listing into active row
25	feature	Implement ISA toggle with instant state reset	completed	Successfully switch between RISC-V stateless to HYMN stateful architecture
26	feature	Implement console output display for HYMN WRITE and RISC-V ecall output	completed	Successfully delivers a correct output at halt
27	feature	Implement import .txt and export source buttons	completed	Successfully able to import a text file straight to the code editor and export a text file of everything in the code editor
28	feature	define FastAPI endpoints: /api/hymn/assemble, /api/hymn/step, /api/riscv/assemble,	completed	Allow the frontend to connect to assemble, and step for each ISA
29	feature	Add a playback speed slider	completed	Changes the program clock speed anywhere from 50ms to 500ms per clock cycle
30	feature	Initial plain HTML/JS prototype (script.js, styles.css) replaced by React	incomplete	removed

Item 31 was removed from the backlog. The original plan was to use an HTML/JS prototype (frontend/script.js, frontend/styles.css), but this was dropped. Once the

React/TypeScript implementation was agreed upon. This decision was made to support component-based architecture with TypeScript type safety.

Sprint 3: RISC-V Assembler Implementation (Report 2 – Final) March 20 - May

Backlog Item ID	**Backlog item type**	**Description**	**Status**	**Acceptance Criteria**
31	testing	Final integration testing of frontend to backend round-trip for both ISAs	completed	Frontend correctly simulates what occurs in the
31	feature	Implement console input display for HYMN READ	completed	Allows the user to use programs where input integers or floats are needed
32	UI	Integrate Figma-sourced SVG icons (logo, play, forward, backward, reset)	completed	Successfully integrated the icons
33	UI	Add cell change animation (flash highlight when memory/register value updates)	completed	When a memory cell is changed, it flashes for 1.5 seconds
34	UI	Add active row highlighting in the Results panel (tracks current PC)	completed	The Results panel highlights which instruction is being executed
35	UI	Add pseudo-instruction expansion markers in the Results listing	completed	The Results panel expands an instruction into either one or two lines, depending on the pseudo-instruction
36	setup	Write/update all README files (main, backend, frontend, HYMN, RISC-V)	completed	The user can understand and use AssemblyViz after reading
37	infra	CI/CD pipeline setup (GitHub Actions for automated test runs)	incomplete	Program automatically runs tests on GitHub after changes are made
38	infra	Containerization for deployment	incomplete	Program can successfully

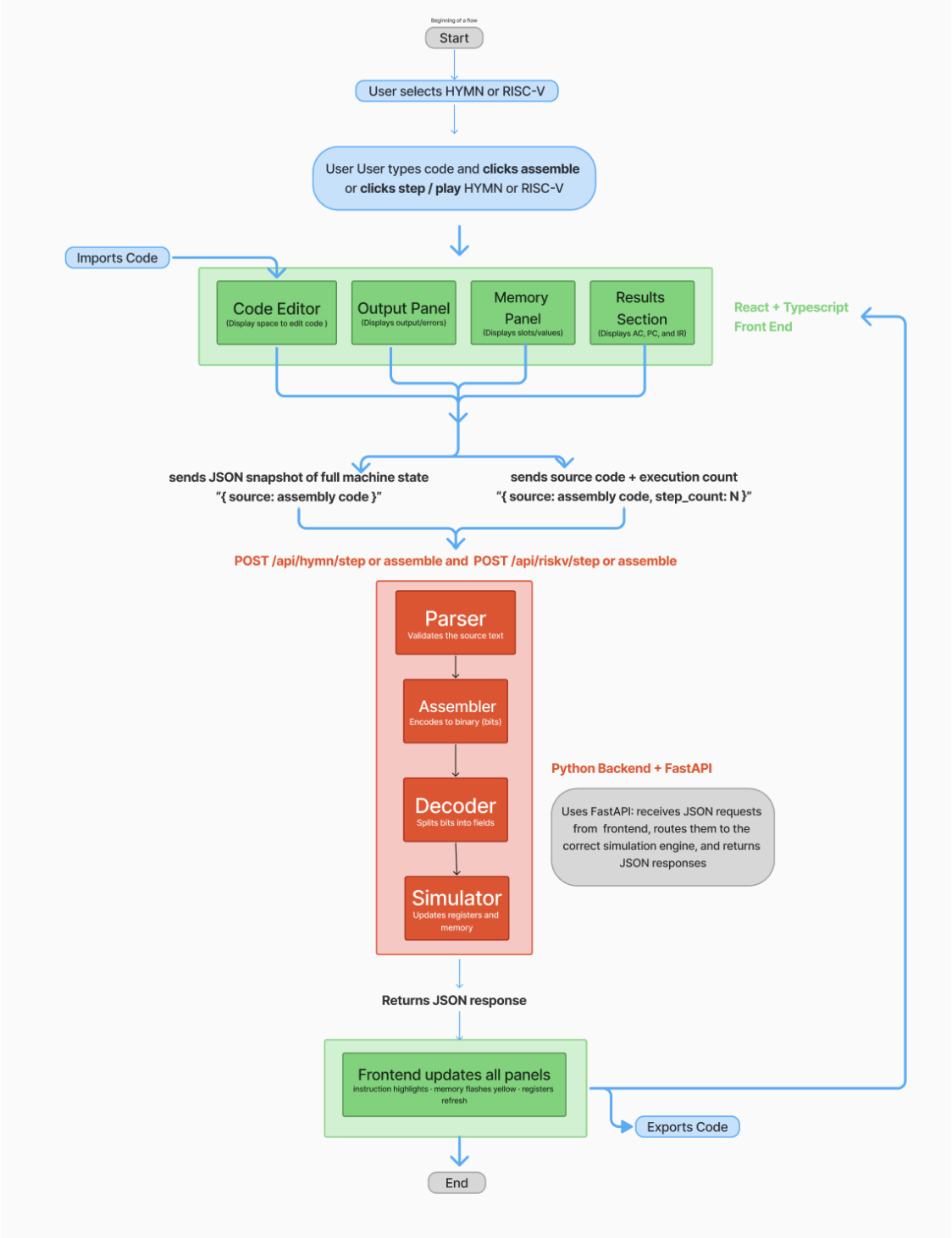
Items 41 and 42 Not Completed. Item 41 Deprioritized due to time constraints near the end of the semester. The test suite runs locally but is not automated on push.
Item 42 Dropped due to time constraints. Our hope is that the simplicity of this project will allow it to be hosted easily if AssemblyVis is to be used by students, but this was out of the scope of our current project timeline.

Future Sprint Backlog:

Backlog Item ID	**Backlog item**	**Description**	**Expected Outcome**
39	Choose a hosting platform and architecture	Consult with the Technology & Innovation (T&I) department at Davidson College to decide where and how to host the frontend and backend	Instructions from T&I on how and where to host our application
40	Reserve a domain	With the help of T&I, reserve a domain (assemblyviz.com) for our website purposes	Reserved domain assemblyviz.com
41	Deploy an application	Deploy the frontend and backend	A website reachable from the Internet
42	Add breakpoint debugger logic for HYMN	Hymn folder backend logic includes a system where you could add breakpoints to lines of code to increase debugging efficiency, but it has not been implemented in frontend yet	User can successfully run

Software Design

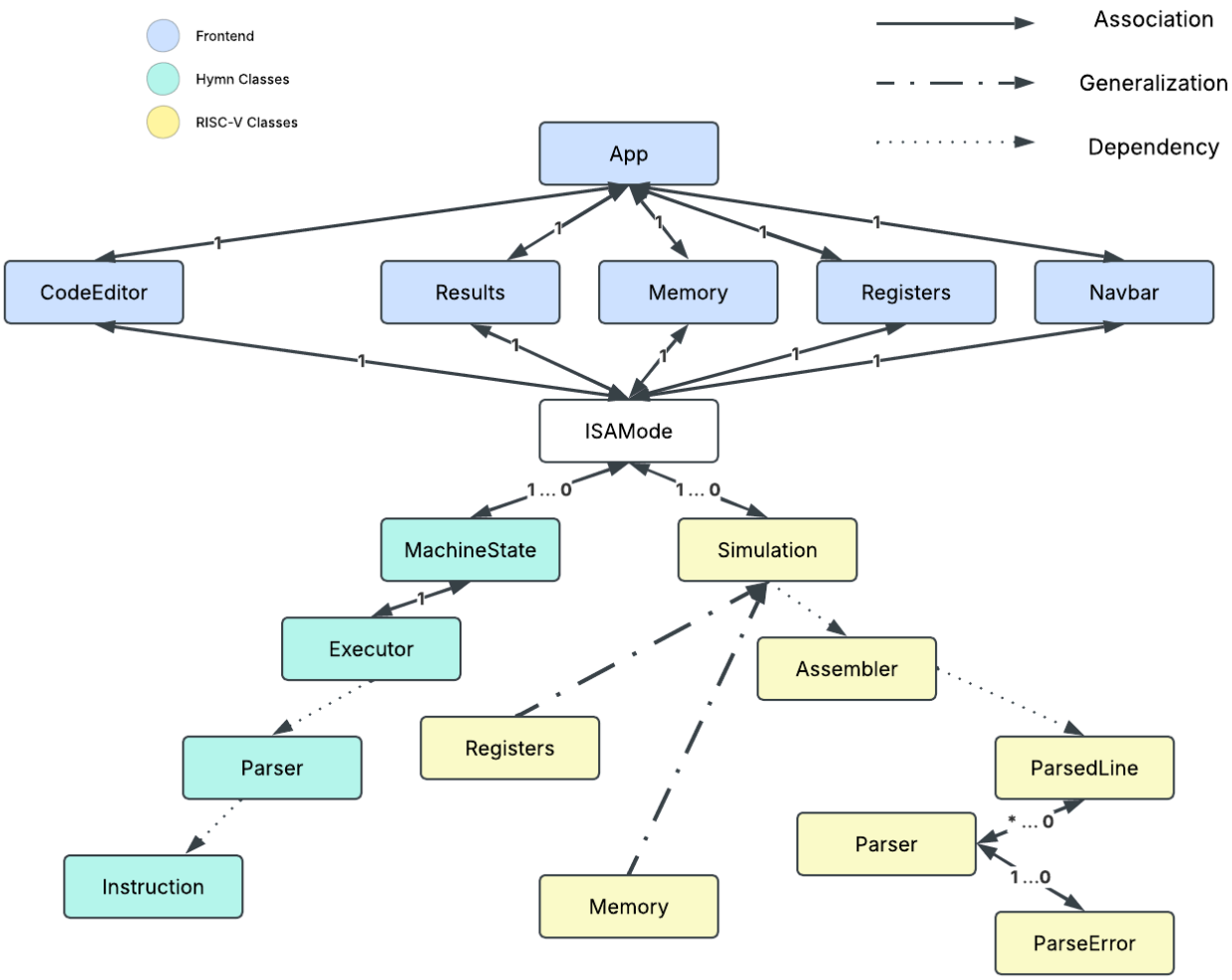
Overall System Architecture:



The diagram shows the flow begins when the user selects the assembly language of choice: HYMN or RISC-V. Then they will write assembly code directly in the code editor. They will click either assemble, step, or play in the front end. All of these commands make the frontend collect the necessary information about the source code and send it to the backend as a JSON object request to one of its endpoints, depending on whether it's an assemble request or a step request. For our project, HYMN is stateful, so the request contains the full machine state {memory[32], pc, ac}. On the other hand, RISC-V is stateless, so it only sends the source code and a step count to execute all the steps again until the specified step count {source, step_count: N}.

The Python backend uses the JSON object received and validates it. There is a pipeline that goes from Parser, Assembler, Decoder, and Simulator. The parser validates that the source text is valid, the assembler encodes it into bits, the decoder separates the bits and decodes each part into executable instructions and values, and the simulator runs the fetch-decode-execute cycle. The backend returns another JSON object to the front-end so that the front-end renders the JSON object appropriately. This return also contains changes in the visual fields that the user will notice: highlighting the active instruction, flashing changed memory cells yellow, and refreshing register values. The system uses no database or authentication. From here, the user can decide to keep iterating and coding or simply end and export the code. In summary, the front-end is rendered first, and the user interacts with the front-end. Then a request is sent back and forth between the backend and front end to allow edits. The system makes sense of the edits and sends them in a way (JSON) that the front-end can render them and display them to the user.

UML Class Diagram



For Frontend:

There is only a 1:1 association between everything in the front end. It is modularized into CodeEditor, where the user types, Results shows the code implementation one clock cycle at a time, memory showing the memory slots, Registers showing what is going through each register, and NavBar where you toggle RISC-V or HYMN. All of these go to the App, where the entire ecosystem comes together through FastAPI. There is no specific class for this, but we added the state, ISAMode (instruction set architecture). The frontend will just receive whatever JSON it's given and doesn't check to see if it's RISC-V or HYMN. If the user is toggled to HYMN, it will run HYMN logic regardless of what is written in input/output, and vice versa for RISC-V. So

ISAMode will receive 1 or 0 MachineStates from HYMN backend or 1 or 0 Simulations from RISC-V based on what is toggled by the user.

For Hymn:

Instruction is the specification of one hymn instruction. Its mnemonic is a 3-bit opcode out of an operand list. Parser does a two-pass regular expression of hymn text into 8-bit machine codes. It depends on the Instruction to check for mnemonics and operands, but it does not store anything long-term. MachineState holds the PC, AC, and IR registers and 32 bytes of memory, and performs the “fetch-decode-execute” cycle. Executor is associated with MachineState 1-to-1. The Executor creates MachineState and fully controls its cycle.

For RISC-V:

Parser produces a list of ParseLine objects. This association is * … 0 because there can be 0 to many lines of parsed code, depending on what is in the code editor. ParseError is returned only if an error is detected, so either 1 or 0. Assembler takes a list of parsed lines as a parameter, reads them, then gets rid of them. This is a dependency. Simulation uses Parse to parse the text and then get rid of it. Simulation never stores anything, so this is a dependency. Assembler is represented inside Simulation and used to encode a ParsedLine into code. This is a dependency. DecodedInstruction is created from a 32-bit code from memory, then used to execute code logic, and then thrown away. This is a dependency. Memory is created inside Simulation and recreated on every reset. Memory cannot exist without simulation, so it is a generalization. Same with registers.

Software Quality

System Testing

The team chose a multi-layered testing approach throughout development, doing testing in each sprint rather than treating it as a final phase. Testing was performed primarily by team members, with feedback from Dr. Hammurabi Mendes and current CSC 250 students during acceptance testing sessions. All four types of testing are unit, integration, system, and acceptance.

Unit testing focused on the Python backend simulation logic for both HYMN and RISC-V. Developers, Nikita and Robby, tested individual stages: the 2-pass parser, the assembler, the decoder, and the simulator, in isolation to verify correctness before connecting them. We considered a unit test successful if it could produce the expected results, decoded instruction, register state, or memory value with no errors.

Integration testing verifies the communication between the Python backend and the React/TypeScript frontend. After each sprint, our developers, Nikita and Robby, confirmed that snapshots returned from the Python simulator (containing register states, memory values, and execution position) were correctly displayed by the four frontend components: the Navbar, Code Editor, Results Panel, and Register Panel. The team ran some example programs for HYMN and RISC-V: a bubble sort, a Fibonacci sequence generator, and a register loop. Integration was considered successful when display panels reflected the correct simulation state simultaneously after each instruction execution.

System testing was conducted by team members who used the full application end-to-end as a student user would. We wrote assembly programs in the code editor, assembled them, and stepped through execution using the play, step forward, step back, and pause control buttons. We verified that the register panel and memory viewer updated correctly after each instruction, and that memory history highlights reflected cells, and that the debugger surfaced meaningful error messages on unrecognized instructions. To evaluate performance, we measured execution time for RISC-V programs of 5, 10, 20, 50, and 100 lines, confirming that the simulator scaled predictably. We have not used an automated system testing framework; all system testing was performed manually in the browser.

Acceptance testing involved sharing the deployed application with Dr. Hammurabi Mendes, Dr. Terrence Lim, current CSC 250 students, and past CSC 250 students. Users were asked to write and step through a simple assembly program, observe register and memory updates, and report whether the interface was clear and easy to navigate without guidance. Dr. Hammurabi Mendes stated that HYMN and RISC-V were correct compared to the documentation. We considered acceptance tests successful when users can complete a task (writing and running the code step by step) without external help, and when Dr. Hammurabi Mendes confirmed simulation behaviour matched the expected assembly behaviour.

Security

AssemblyViz does not implement user authentication or store any user data, and this is an intentional design decision consistent with the project's educational scope. The system follows a stateless client-server model: the React frontend transmits source code (and, for HYMN, the current machine state) to the FastAPI backend via POST requests to endpoints such as /api/hymn/step and /api/riscv/step. The backend processes each request and returns a JSON snapshot of the resulting simulation state, and no data is persisted between requests. There is no database, no session store, and no logging of user-submitted code. Each request is handled independently and discarded once the response is returned.

The primary security consideration for AssemblyViz is the handling of user-supplied assembly code, which is parsed and executed by the Python backend. Since the simulator executes instructions in a controlled virtual environment rather than on the host system, there is no risk of arbitrary code execution or system compromise from malformed user input. The parser rejects unrecognized instructions and registers and returns clear error messages to the frontend without attempting to execute invalid code. No passwords, credentials, or personally identifiable information are collected or transmitted at any point in the system, consistent with the project’s design goal of being a lightweight, privacy-preserving educational tool that requires no login.

Project Retrospective

- **What went well?**
  - Concurrent development of HYMN and RISC-V ISA's. At the beginning of development, we focused on supporting HYMN first by using XP conventions such as pair programming. However, we soon realized that paired programming was the source of inefficiency since progress was too slow with only one ISA being developed at a time. Thus, we quickly switched to independent development, where one developer is responsible for HYMN and another for RISC-V.
  - Fast development of the backend. We were able to set up the most important parts, such as parser, assembler, decoder, and simulator, of both ISA's in the first month but without debugging. This implementation, though lacking some features, gave us much room for testing and improvement–we verified that basic functionality worked correctly and then switched to incorporating the debugging feature.
  - Quick switch from HTML + JavaScript to TypeScript + React. The frontend team initially created a prototype of the website using HTML and JavaScript, but then switched to TypeScript and React relatively quickly to improve modularization and software optimization. For example, React's component structure made it easier to build the separate panels from Requirement 1.
- **What didn't go well, and what should we have done differently?**
  - Poor estimation of frontend development time. Our team was supposed to work on the frontend and backend in parallel. While the backend was completed relatively quickly, the frontend took longer than expected due to the mid-project switch from JavaScript to TypeScript and insufficient planning.
  - Mid-project technology switch on the frontend. While moving to TypeScript and React ultimately improved the codebase, it introduced rework and contributed to the frontend falling behind schedule. In hindsight, we should have evaluated and committed to our technology stack during the planning phase before development began.
- **What should we have done differently?**
  - We would focus on better sprint planning to account for potential technical difficulties and ensure tasks are completed by the deadline.
  - We should have evaluated and committed to our technology stack during the planning phase before development began.